

## Trabalho 01

Tradutor Dirigido por Sintaxe: de Infixa para Pós-fixa

Autor: Tony Esaú de Oliveira

### 1 Parte A: projeto da tradução

#### 1.1 Item 1: eliminação da recursão à esquerda

A gramática de partida é recursiva à esquerda em *expr*, *term* e *num*:

$$\begin{aligned} \text{expr} &\rightarrow \text{expr} + \text{term} \mid \text{expr} - \text{term} \mid \text{term} \\ \text{term} &\rightarrow \text{term} * \text{fact} \mid \text{term} / \text{fact} \mid \text{fact} \\ \text{fact} &\rightarrow ( \text{expr} ) \mid \text{num} \\ \text{num} &\rightarrow \text{num digit} \mid \text{digit} \\ \text{digit} &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \end{aligned}$$

Aplicando a técnica  $A \rightarrow A\alpha \mid A\beta \mid \gamma \Rightarrow A \rightarrow \gamma A', A' \rightarrow \alpha A' \mid \beta A' \mid \epsilon$ , com um novo não-terminal para cada não-terminal recursivo (nomeado com notação linha '):

- em *expr*:  $\alpha = + \text{term}$ ,  $\beta = - \text{term}$ ,  $\gamma = \text{term}$ ;
- em *term*:  $\alpha = * \text{fact}$ ,  $\beta = / \text{fact}$ ,  $\gamma = \text{fact}$ ;
- em *num*:  $\alpha = \text{digit}$ ,  $\gamma = \text{digit}$

O enunciado pede a transformação apenas de *expr* e *term*; a de *num* foi feita também, para que a gramática inteira fique sem recursão à esquerda. A gramática resultante é:

$$\begin{aligned} \text{expr} &\rightarrow \text{term expr}' \\ \text{expr}' &\rightarrow + \text{term expr}' \mid - \text{term expr}' \mid \epsilon \\ \text{term} &\rightarrow \text{fact term}' \\ \text{term}' &\rightarrow * \text{fact term}' \mid / \text{fact term}' \mid \epsilon \\ \text{fact} &\rightarrow ( \text{expr} ) \mid \text{num} \\ \text{num} &\rightarrow \text{digit num}' \\ \text{num}' &\rightarrow \text{digit num}' \mid \epsilon \\ \text{digit} &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \end{aligned}$$

#### 1.2 Item 2: esquema de tradução para a notação pós-fixada

As ações semânticas são escritas entre chaves no corpo das produções e executadas no momento em que o analisador passa por elas, da esquerda para a direita:

$$\begin{aligned}
expr &\rightarrow term\ expr' \\
expr' &\rightarrow +\ term\ \{\text{print}('+\')\}\ expr' \\
&\quad | -\ term\ \{\text{print}('-'\')\}\ expr' \\
&\quad | \epsilon \\
term &\rightarrow fact\ term' \\
term' &\rightarrow *\ fact\ \{\text{print}('*'\')\}\ term' \\
&\quad | /\ fact\ \{\text{print}('/'\')\}\ term' \\
&\quad | \epsilon \\
fact &\rightarrow (\ expr\ )\ |\ num \\
num &\rightarrow digit\ num' \\
num' &\rightarrow digit\ num'\ |\ \epsilon \\
digit &\rightarrow 0\ \{\text{print}('0'\')\}\ |\ 1\ \{\text{print}('1'\')\}\ |\ \dots\ |\ 9\ \{\text{print}('9'\')\}
\end{aligned}$$

Posição de cada ação:

- Nas produções de  $expr'$  e  $term'$ , a ação do operador fica **depois do segundo operando** ( $term$  ou  $fact$ ) e **antes da chamada recursiva** ( $expr'$  ou  $term'$ ). Assim o operador é impresso logo após os seus dois operandos e antes de tratar o operador seguinte, o que produz a associatividade à esquerda.
- Em  $digit$ , a ação fica logo após o terminal e imprime o próprio dígito, de modo que os dígitos de um número saem na ordem em que aparecem.
- As produções de  $expr$ ,  $term$ ,  $fact$ ,  $num$ ,  $num'$  e as alternativas  $\epsilon$  não têm ação: os parênteses não aparecem na notação pós-fixada.

Na implementação,  $num$  é reconhecido pelo procedimento `proximo` como um único token. As ações de  $digit$  equivalem, então, a uma única ação  $fact \rightarrow num\ \{\text{print}(num.valor)\}$ , que imprime o número inteiro de uma vez. É essa forma que aparece nas árvores do item 4.

### 1.3 Item 3: conjuntos FIRST e justificativa do analisador preditivo

| Não-terminal | Alternativa      | FIRST                              |
|--------------|------------------|------------------------------------|
| $expr$       | $term\ expr'$    | $\{ (, 0, \dots, 9 \}$             |
| $expr'$      | $+\ term\ expr'$ | $\{ + \}$                          |
|              | $-\ term\ expr'$ | $\{ - \}$                          |
|              | $\epsilon$       | $\{ \epsilon \}$                   |
| $term$       | $fact\ term'$    | $\{ (, 0, \dots, 9 \}$             |
| $term'$      | $*\ fact\ term'$ | $\{ * \}$                          |
|              | $/\ fact\ term'$ | $\{ / \}$                          |
|              | $\epsilon$       | $\{ \epsilon \}$                   |
| $fact$       | $(\ expr\ )$     | $\{ ( \}$                          |
|              | $num$            | $\{ 0, \dots, 9 \}$                |
| $num$        | $digit\ num'$    | $\{ 0, \dots, 9 \}$                |
| $num'$       | $digit\ num'$    | $\{ 0, \dots, 9 \}$                |
|              | $\epsilon$       | $\{ \epsilon \}$                   |
| $digit$      | $0, 1, \dots, 9$ | $\{ 0 \}, \{ 1 \}, \dots, \{ 9 \}$ |

Os conjuntos que não são imediatos:  $FIRST(digit\ num') = FIRST(digit) = \{0, \dots, 9\}$ , pois  $digit$  não gera  $\epsilon$ ;  $FIRST(fact\ term') = FIRST(fact) = \{ ( \} \cup FIRST(num) = \{ (, 0, \dots, 9 \}$ , pois  $fact$  não gera  $\epsilon$ ; e, pelo mesmo motivo,  $FIRST(term\ expr') = FIRST(term) = \{ (, 0, \dots, 9 \}$ .

**Por que um analisador preditivo pode ser usado.** Um analisador preditivo escolhe a produção olhando apenas o próximo símbolo da entrada, sem retrocesso. Isso é possível porque a gramática

do item 1 satisfaz três condições:

1. **Não há recursão à esquerda:** toda chamada recursiva acontece depois que algum terminal foi consumido, então o analisador não entra em laço infinito.
2. **Para cada não-terminal, os FIRST das alternativas são disjuntos:**  $\{+\} \cap \{-\} = \emptyset$  em  $expr'$ ;  $\{\star\} \cap \{/ \} = \emptyset$  em  $term'$ ;  $\{( \} \cap \{0, \dots, 9\} = \emptyset$  em  $fact$ ; e cada alternativa de  $digit$  tem um dígito diferente. Os demais não-terminais têm uma única alternativa. Logo, cada símbolo de lookahead indica no máximo uma produção.
3. **As alternativas  $\epsilon$  são escolhidas sem ambiguidade:** em  $expr'$ ,  $term'$  e  $num'$ , o analisador escolhe  $\epsilon$  sempre que o lookahead não pertence ao FIRST das outras alternativas daquele não-terminal. Como essas outras alternativas começam com terminais bem definidos (+ ou -;  $\star$  ou /; um dígito), qualquer outro símbolo indica que aquele nível da expressão terminou, e a decisão é tomada sem consumir entrada e sem precisar voltar atrás.

Portanto, a gramática é LL(1): em todo ponto da análise, um único símbolo de lookahead determina sem ambiguidade a produção a usar, e um analisador descendente recursivo preditivo pode ser empregado.

#### 1.4 Item 4: árvores de derivação com as ações semânticas

Nas árvores abaixo, as ações semânticas aparecem como filhos extras, ligadas por arestas tracejadas, e o número entre parênteses indica a ordem em que cada ação é executada na busca em profundidade (os filhos de cada nó são visitados da esquerda para a direita, e uma ação é executada no momento em que é visitada). Como explicado no item 2, a subárvore de  $num$  foi abreviada para a ação que imprime o número.

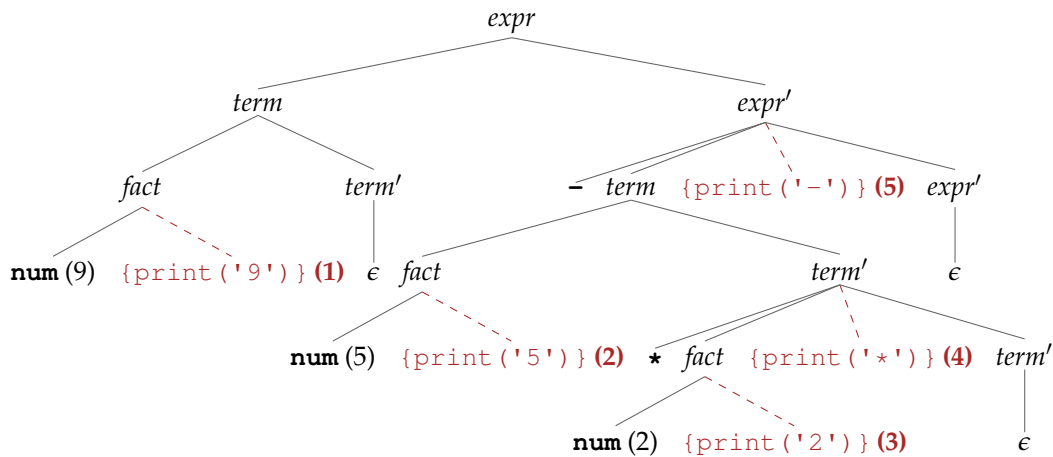


Figura 1: Árvore de derivação de  $9-5*2$ . Ordem das ações: 9, 5, 2,  $\star$ , -. Saída: 9 5 2  $\star$  -.

**Comparação.** As duas cadeias têm os mesmos números e operadores, mas as árvores têm formas diferentes, e a forma da árvore determina a ordem das ações.

Em  $9-5*2$ , a multiplicação fica *dentro* do  $term$  que é o segundo operando da subtração: o nó  $term'$  que contém  $\star$  é descendente do  $term$  pendurado em  $expr'$ . Na busca em profundidade, essa subárvore é visitada por completo, inclusive a ação de  $\star$  (ação 4), antes que se chegue à ação de - (ação 5), que é irmã desse  $term$  e está à sua direita. Por isso a saída é 9 5 2  $\star$  -: a precedência de  $\star$  sobre - resulta da estrutura da gramática, em que  $\star$  é tratado num nível mais profundo ( $term$ ) do que - ( $expr$ ).

Em  $(9-5)*2$ , os parênteses fazem com que toda a subtração fique dentro de um único  $fact$ , pela produção  $fact \rightarrow (expr)$ . Esse  $fact$  é o primeiro operando da multiplicação, então a ação de - (ação 3) é executada ao percorrer essa subárvore, antes de se visitar o  $term'$  que contém a multiplicação e a sua ação (ação 5). A saída é 9 5 - 2  $\star$ . Em ambos os casos, os terminais ( e ) não têm

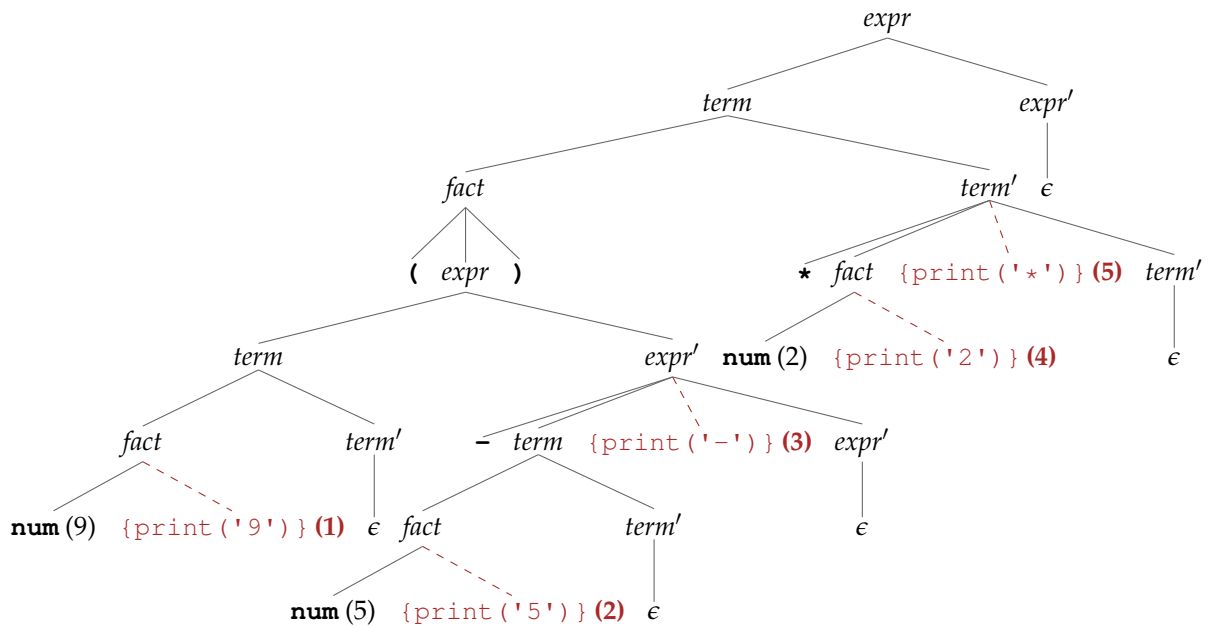


Figura 2: Árvore de derivação de  $(9-5)*2$ . Ordem das ações: 9, 5, -, 2, \*. Saída: 9 5 - 2 \*.

ação, o que confirma que os parênteses só afetam a forma da árvore e não aparecem na notação pós-fixada.

## 2 Decisões de implementação

O tradutor (`tradutor.py`) usa apenas a biblioteca padrão do Python e é um analisador descendente recursivo preditivo escrito à mão, com um procedimento por não-terminal da gramática do item 1.

- **Procedimentos.** Os não-terminais *expr*, *expr'*, *term*, *term'* e *fact* correspondem às funções `expr`, `expr_linha`, `term`, `term_linha` e `fact` (o apóstrofo não é permitido em nomes do Python). Cada função percorre o corpo da produção da esquerda para a direita: terminal vira uma chamada de `casar` (`match`), não-terminal vira a chamada da função correspondente, e ação semântica vira uma chamada de `emitir`. A escolha da produção é feita com `if/elif` sobre o lookahead, usando os conjuntos FIRST do item 3, e a alternativa `ε` corresponde a não fazer nada.
- **casar (o match).** Compara o tipo do token atual com o tipo esperado; se forem iguais, obtém o próximo token com `proximo`, senão lança um erro de sintaxe.
- **proximo.** Faz o papel de analisador léxico: ignora espaços e tabulações, agrupa dígitos consecutivos num único token `num` e reconhece operadores e parênteses. Por isso, `num`, `num'` e `digit` não têm procedimentos próprios. É chamado sob demanda, um token por vez, a cada `casar` bem-sucedido.
- **Tokens.** Cada token é uma tupla (`tipo`, `valor`), por exemplo `("num", "10")` ou `("+", "+")`. O analisador compara o tipo; a tradução usa o valor. O fim da linha é representado por um token `("fim", "")`.
- **Estado.** A linha atual, a posição de leitura, o lookahead e a saída ficam em variáveis globais do módulo, compartilhadas pelas funções.

- **Saída.** Em vez de imprimir cada símbolo diretamente, a ação `emitir` acrescenta o símbolo a uma lista, e ao final da linha a lista é impressa unida por espaços. Isso garante exatamente uma linha de saída por expressão, com os tokens separados por um espaço.
- **Fim da expressão.** Depois de `expr()`, verifica-se se o `lookahead` é o token de fim. Se ainda sobrou entrada (por exemplo, em `9 5`), a linha não é uma expressão válida como um todo.
- **Erros.** Erros léxicos e sintáticos lançam `SyntaxError`, que é capturado na `main` para cada linha. Uma linha inválida produz uma linha de saída `erro: <mensagem>` e o programa continua nas linhas seguintes, mantendo a correspondência de uma linha de saída por linha de entrada. O enunciado não especifica o comportamento para entradas inválidas, por isso essa foi uma decisão própria.
- **Entrada.** O programa lê do arquivo passado como argumento (`python3 tradutor.py entradas.txt`) ou, sem argumento, da entrada padrão (`python3 tradutor.py <entradas.txt>`), já que o enunciado menciona as duas formas. Linhas em branco são ignoradas.

Trecho que mostra a correspondência entre a gramática e o código:

```
def term_linha():
    # term' * fact {emitir('*')} term'
    if lookahead[0] == "*":
        casar("*")
        fact()
        emitir("*")
        term_linha()
    # term' / fact {emitir('/') } term'
    elif lookahead[0] == "/":
        casar("/")
        fact()
        emitir("/")
        term_linha()
    # term' vazio : não consome nada.

def fact():
    # fact ( expr )
    if lookahead[0] == "(":
        casar("(")
        expr()
        casar(")")
    # fact num {emitir(valor do número)}
    elif lookahead[0] == "num":
        emitir(lookahead[1]) # emite antes de casar, que troca o lookahead
        casar("num")
    else:
        raise SyntaxError(...)
```

### 3 Testes

Além dos 7 casos públicos do enunciado, foram criados casos próprios (arquivos `txt` na pasta `/testes`). Todos os casos válidos produzem a saída esperada.

